

LABORATORY

LAB 10

RFID Card Detection with MFRC522

Reading Card UIDs with the RC522 Device API in EduFramework

S32K144 · MaaZEDU

Contents

1	Introduction	2
1.1	Overview	2
1.2	Objectives	2
2	Background	2
2.1	RFID Reader, Card/Tag, and UID	2
2.2	SPI Communication with the MFRC522	3
2.3	From Card Detection to UID Reading	3
3	Hardware Setup	3
3.1	Required Hardware	3
3.2	Pin Mapping	4
3.3	MFRC522 Connection	4
4	EduFramework API	5
4.1	<code>rc522_uid_t</code> Structure	5
4.2	<code>RC522_PCD_Init()</code>	5
4.3	<code>RC522_PICC_IsNewCardPresent()</code>	6
4.4	<code>RC522_PICC_ReadCardSerial()</code>	6
4.5	<code>RC522_PICC_HaltA()</code>	7
5	Lab Exercise	7
5.1	Requirements	7
5.2	Program	7
5.3	Verification	9
6	Extension	9
6.1	Checking <code>VersionReg</code> with <code>RC522_PCD_GetVersion()</code>	9
6.2	Identifying an Authorized Card by UID	10
6.3	LED Control Based on the Identification Result	10
7	References	10

1 Introduction

1.1 Overview

RFID (Radio Frequency Identification) allows a system to identify cards or tags through radio communication without requiring direct electrical contact. In this lab, the MFRC522 module is used as a reader to detect compatible RFID cards and obtain their identification information.

[1].

In a basic RFID application, the reader detects a card, reads its UID, and then passes the UID to the program for display or processing. The Alfaisal University lab uses the same model: the MFRC522 reads the card UID, and the application can compare that UID with a predefined list to make a decision [2].

1.2 Objectives

OBJECTIVES

The objectives of this laboratory are:

1. Describe the roles of the RFID reader, RFID card/tag, and UID in a basic identification system.
2. Explain the roles of the SPI signals `SCK`, `MOSI`, `MISO`, and chip-select when connecting the MFRC522 to a microcontroller.
3. Describe the processing flow from card appearance until the UID is read by the RC522 Device API.
4. Use `RC522_PCD_Init()` to initialize the reader; use `RC522_PICC_IsNewCardPresent()` and `RC522_PICC_ReadCardSerial()` to detect a card and read its UID.
5. Build and verify an application that displays the UIDs of multiple RFID cards on the Serial Monitor.

2 Background

2.1 RFID Reader, Card/Tag, and UID

The MFRC522 is a reader/writer IC designed for contactless communication at `13.56 MHz`. The IC supports communication with cards and transponders according to ISO/IEC 14443 A, MIFARE, and NTAG. The transmitter section of the MFRC522 drives the reader antenna, while the receiver section receives and decodes the response signal from the card [1].

In this lab, the MFRC522 acts as the reader, while the RFID card or tag is the object placed within the reading area. The application does not directly process the radio-frequency signal; instead, the reader communicates with the card and provides data to the microcontroller through its host interface [1].

An important piece of information when working with RFID cards is the UID. A UID is a unique sequence of bytes used to uniquely identify an RFID card. For example, a four-byte UID can be represented as `BD 31 15 2B` [2]. The current RC522 Device implementation in EduFramework supports UIDs with a size of `4 bytes`. After a successful read, the UID bytes are stored in `rc522_uid_t`, together with the `size` field, which indicates the number of valid bytes, and the `sak` field, which contains the Select Acknowledge value [4].

Note

The current RC522 Device implementation supports only four-byte UIDs. Cascade UIDs with sizes of 7 bytes or 10 bytes, together with MIFARE authentication and block read/write functions, are not currently supported by the Device [4].

2.2 SPI Communication with the MFRC522

The MFRC522 supports multiple host interfaces, including SPI. When SPI is used, the MFRC522 operates as a slave device; the microcontroller generates the `SCK` clock signal, transmits data to the MFRC522 through `MOSI`, and receives data from the MFRC522 through `MISO` [1].

In addition to these three signal lines, SPI requires a device-select signal. The MFRC522 datasheet denotes this signal as `NSS`; it must remain low while a data stream is being transferred [1]. On the MFRC522 module used in this lab, the chip-select pin is commonly labeled `SDA/SS`. When the module operates in SPI mode, this pin is controlled by the RC522 Device as an active-low chip-select rather than being used as an I2C data line [4]. The communication path between MaaZEDU and the reader can be described as:

MaaZEDU ↔ SPI ↔ MFRC522 ↔ RF ↔ RFID card/tag

The RC522 Device uses the fixed EduFramework SPI lines for `SCK`, `MOSI`, and `MISO`, while allowing the application to select one Digital Logical Pin as the chip-select pin and another Digital Logical Pin as the reset pin [4].

2.3 From Card Detection to UID Reading

At the application level, the process of reading a card can be divided into three main steps: initialize the reader, check whether a new card is present, and read the card UID. The same organization is used in the Alfaisal University RFID lab, where the program first initializes SPI and the MFRC522, then checks for a new card and reads its UID [2].

In EduFramework, `RC522_PCD_Init()` performs the reader initialization. The API configures the two control pins, initializes SPI in master mode, resets the MFRC522, applies the default configuration, enables the antenna, and validates communication through `VersionReg` [4].

After the reader is ready, `RC522_PICC_IsNewCardPresent()` sends a request to check for a new ISO/IEC 14443A card within the reading area. If a valid response is received, `RC522_PICC_ReadCardSerial()` performs the required operations to select the card and stores the four-byte UID in an `rc522_uid_t` structure [4].

This lab does not require implementing the low-level protocol steps. The application only uses the boolean results of these two APIs to determine when the UID is ready to be displayed.

The processing flow of the main exercise is:

Card appears → `IsNewCardPresent()` → `ReadCardSerial()` → UID bytes → `Serial1` → `Serial Monitor`

After processing a card, `RC522_PICC_HaltA()` sends the HALT command to the selected card. The RC522 Device treats the timeout after the HALT command as the expected behavior of a card that has entered the halt state [4].

3 Hardware Setup

3.1 Required Hardware

The lab uses a MaaZEDU Development Board, an MFRC522 module, and at least one compatible RFID card or tag. The MFRC522 communicates with the microcontroller through SPI, while the Serial Monitor is used to observe the UID that has been read.

Hardware	Description
MaaZEDU Development Board	Development board using the S32K144 microcontroller.
MFRC522 RFID reader module	Reader module used to detect and communicate with compatible RFID cards at 13.56 MHz .
RFID card/tag	Card or tag used to verify card detection and UID reading.
Jumper wires	Connect power, SPI, chip-select, and reset between the MFRC522 and MaaZEDU.
USB Cable	Connect the board to a computer for power, programming, and Serial Monitor access.

Table 10.3.1. Hardware used in the lab

3.2 Pin Mapping

The RC522 Device uses the EduFramework SPI lines for clock and data. In this lab, `GPI00` is selected as the software chip-select and `GPI01` is selected as the reset pin. On MaaZEDU, the SPI signals correspond to the board SPI pin group; `GPI00` maps to `PTE0` and `GPI01` maps to `PTD17` [3] [4] .

Component	Logical Pin	MCU Pin	Function
MFRC522 SCK	<code>SPI_SCK</code>	<code>PTB14</code>	SPI Clock
MFRC522 MOSI	<code>SPI_SOUT</code>	<code>PTB16</code>	SPI Data: MaaZEDU → MFRC522
MFRC522 MISO	<code>SPI_SIN</code>	<code>PTB15</code>	SPI Data: MFRC522 → MaaZEDU
MFRC522 SDA/SS	<code>GPI00</code>	<code>PTE0</code>	Software Chip-Select
MFRC522 RST	<code>GPI01</code>	<code>PTD17</code>	Digital Output / Reset

Table 10.3.2. MFRC522 signal mapping used in the lab

3.3 MFRC522 Connection

In the verified configuration used for this lab, the MFRC522 is powered from `3.3V` , shares `GND` with MaaZEDU, and is connected through the SPI lines shown in the mapping table above. The `IRQ` pin is not used in this lab.

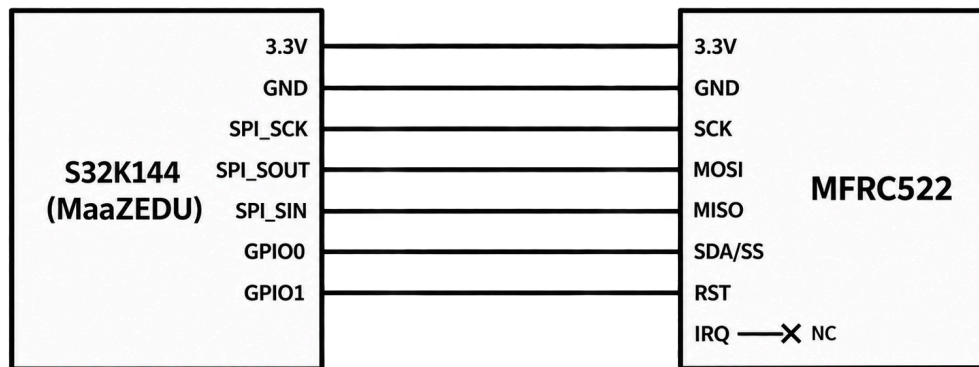


Figure 10.3.1. MFRC522 connection to the MaaZEDU Development Board

Note

The pin labeled `SDA/SS` on the module is used as the chip-select when working with the RC522 Device through SPI. Do not connect this pin to the MaaZEDU I2C interface. `GPIO0` and `GPIO1` are the two Logical Pins selected for chip-select and reset in this lab [4].

4 EduFramework API

The main exercise uses four APIs from the RC522 Device: `RC522_PCD_Init()`, `RC522_PICC_IsNewCardPresent()`, `RC522_PICC_ReadCardSerial()`, and `RC522_PICC_HaltA()`. Reader reset, antenna control, and `VersionReg` reading operations are reserved for the Extension section [4].

4.1 rc522_uid_t Structure

UID data is stored in the `rc522_uid_t` structure. For this lab, the two fields of interest are `bytes`, which stores each UID byte, and `size`, which indicates the number of valid UID bytes. The `sak` field is stored by the library but does not need to be processed in the main exercise [4].

Example declaration:

```
rc522_uid_t uid = {{0U}, 0U, 0U};
```

After a successful read, the UID bytes can be accessed through `uid.bytes[i]`, where `i` ranges from `0` to `uid.size - 1`.

4.2 RC522_PCD_Init()

`RC522_PCD_Init()` initializes the reader with one Logical Pin as chip-select and another Logical Pin as reset. The API returns `true` when initialization and communication validation complete successfully [4].

RC522_PCD_Init

Initialize the MFRC522, SPI, and the control pins required by the reader.

Syntax

```
RC522_PCD_Init(csPin, resetPin);
```

Parameters

Parameter	Type / accepted value	Description
csPin	Digital Logical Pin	Digital pin used as the MFRC522 software chip-select.
resetPin	Digital Logical Pin	Digital pin used to reset the MFRC522.

Return value

`true` when initialization and communication validation succeed; `false` when pin configuration, reset, or communication validation fails.

Example:

```
bool ready = RC522_PCD_Init(GPI00, GPI01);
```

4.3 RC522_PICC_IsNewCardPresent()

`RC522_PICC_IsNewCardPresent()` checks whether a new ISO/IEC 14443A card responds within the reading area. The application uses the return value to avoid attempting a UID read when no card is present [4].

RC522_PICC_IsNewCardPresent

Check for the presence of a new card within the reading area.

Syntax

```
RC522_PICC_IsNewCardPresent();
```

Return value

`true` when a valid card response is received; `false` when no card is present or communication is unsuccessful.

Example:

```
if (true == RC522_PICC_IsNewCardPresent())
{
    /* A new card is present. */
}
```

4.4 RC522_PICC_ReadCardSerial()

`RC522_PICC_ReadCardSerial()` reads and selects the current card, then stores the UID in the supplied structure. The current implementation accepts only four-byte UIDs [4].

RC522_PICC_ReadCardSerial

Read the UID of the detected card and store the result in the UID structure.

Syntax

```
RC522_PICC_ReadCardSerial(&uid);
```

Parameters

Parameter	Type / accepted value	Description
pUid	UID data	Address of the <code>rc522_uid_t</code> structure used to receive the UID bytes, UID size, and SAK.

Return value

`true` when the UID and SAK are read successfully; `false` when the argument is invalid, communication fails, or the card requires an unsupported UID cascade level.

Example:

```
if (true == RC522_PICC_ReadCardSerial(&uid))
{
    Serial1_printInt((int)uid.bytes[0]);
}
```

4.5 RC522_PICC_HaltA()

After the UID has been processed, `RC522_PICC_HaltA()` sends the HALT command to the selected card. The API has no parameters and returns no value [4].

RC522_PICC_HaltA

Put the selected card into the HALT state after processing is complete.

Syntax

```
RC522_PICC_HaltA();
```

Return value

No return value.

Example:

```
RC522_PICC_HaltA();
```

5 Lab Exercise**5.1 Requirements**

Build a program that detects RFID cards with the MFRC522 and displays each card UID on the Serial Monitor. The module uses `GPIO0` as chip-select, `GPIO1` as reset, and the `SPI_SCK`, `SPI_SOUT`, and `SPI_SIN` lines for SPI communication.

The program initializes `Serial1` at a baud rate of `9600`. When the MFRC522 is initialized successfully, the Serial Monitor displays `RC522 ready.`. In the main loop, the program reads a UID only when a new card is detected. Each UID byte is printed in decimal format.

Use at least two cards or tags, if available, to verify that the program can read and display different UIDs.

5.2 Program

In `src/main.c`, implement the hardware-verified program as follows:

```
#include "Arduino.h"
#include "rc522.h"

int main(void)
{
    rc522_uid_t uid = {{0U}, 0U, 0U};
    uint8_t i = 0U;

    setup();

    Serial1_begin(9600U);

    if (true == RC522_PCD_Init(GPI00, GPI01))
    {
        Serial1_println("RC522 ready.");
    }
    else
    {
        Serial1_println("RC522 initialization failed.");
    }

    while (1)
    {
        if (true == RC522_PICC_IsNewCardPresent())
        {
            if (true == RC522_PICC_ReadCardSerial(&uid))
            {
                Serial1_print("UID: ");

                for (i = 0U; i < uid.size; i++)
                {
                    Serial1_printInt((int)uid.bytes[i]);
                    Serial1_print(" ");
                }

                Serial1_println("");

                RC522_PICC_HaltA();
                delay(500U);
            }
        }

        delay(20U);
    }

    return 0;
}
```

Listing 10.5.1. Program for detecting cards and displaying UIDs with the MFRC522

`setup()` initializes the EduFramework platform components, and `Serial1_begin(9600U)` prepares the Serial Monitor channel. Then, `RC522_PCD_Init(GPI00, GPI01)` configures the reader with `GPI00` as chip-select and `GPI01` as reset [4] .

In the main loop, the program calls `RC522_PICC_IsNewCardPresent()` first. Only when this API returns `true` does the program call `RC522_PICC_ReadCardSerial(&uid)`. This organization of checking for a card before reading its UID also corresponds to the RFID application flow presented in the Alfaisal University lab [2] [4]. When the UID is read successfully, the `for` loop iterates over `uid.size` elements and prints each byte from `uid.bytes[]` to the Serial Monitor. The lab uses decimal representation to keep the display logic simple. Then, `RC522_PICC_HaltA()` completes the processing of the current card [4].

The program processing flow is:

```
MFRC522 ready → Detect card → Read UID → Print each UID byte → Halt card →  
Wait for the next scan
```

5.3 Verification

Build and upload the program to the MaaZEDU Development Board, then open the Serial Monitor at a baud rate of `9600`. When the reader initializes successfully, observe the message:

```
RC522 ready.
```

Place a card or tag within the MFRC522 reading area. When the read succeeds, the UID is displayed on the Serial Monitor. With the two cards used to verify this lab, the observed output has the following form:

```
UID: 100 29 168 0  
UID: 100 29 168 0  
UID: 238 104 23 5  
UID: 238 104 23 5
```

The two different value groups show that the application received different UID data from the two tested cards/tags. The specific values depend on the card being used and are not fixed in the main program.

EXPECTED RESULT

After the MFRC522 initializes successfully, the Serial Monitor displays `RC522 ready.`. When a compatible card or tag is placed within the reading area, the program detects the card, reads the four-byte UID, and prints the UID bytes to the Serial Monitor. When another card is used, the displayed UID data changes according to the card being read. If the reader does not initialize successfully, the program displays `RC522 initialization failed.` [4].

6 Extension

The main exercise only reads and displays the UID. The RC522 Device also provides APIs for observing reader status, while the UID that has been read can be used as input for a simple identification task [4] [2].

6.1 Checking VersionReg with `RC522_PCD_GetVersion()`

After the reader initializes successfully, `RC522_PCD_GetVersion()` returns the `VersionReg` value read from the MFRC522. The RC522 Device uses this same register during initialization to validate communication with the reader [4].

`RC522_PCD_GetVersion`

Read the MFRC522 `VersionReg` value after the Device has been initialized.

Syntax

```
RC522_PCD_GetVersion();
```

Return value

The `VersionReg` value; `RC522_VERSION_INVALID` if the Device has not been initialized.

Extension: After the `RC522 ready.` message, read `VersionReg` and print its value in decimal format on the Serial Monitor.

Example:

```
Serial1_print("VersionReg: ");  
Serial1_printlnInt((int)RC522_PCD_GetVersion());
```

6.2 Identifying an Authorized Card by UID

The Alfaisal University lab extends UID reading into an access-control task by storing authorized UIDs in advance and comparing the scanned UID with that list [2].

With the current EduFramework implementation, the supported UID size is four bytes. A known UID can be stored in an array and compared byte by byte with `uid.bytes[]`.

Example using a UID measured in this lab:

```
static const uint8_t authorizedUid[4U] =  
{  
    238U, 104U, 23U, 5U  
};
```

Extension requirements:

- When all four bytes of the scanned UID match `authorizedUid`, print `Authorization: GRANTED`.
- When at least one byte is different, print `Authorization: DENIED`.
- Do not change the system state if the UID is not authorized.

This extension adds a processing step after UID reading without changing the communication flow with the MFRC522:

Detect → Read UID → Compare UID → GRANTED / DENIED

6.3 LED Control Based on the Identification Result

After completing the UID comparison, visual feedback can be added using the two LEDs on MaaZEDU. One possible implementation uses the green LED for the authorized state and the red LED for the denied state.

Extension requirements:

- Authorized UID: change the controlled state and update the green LED.
- Unauthorized UID: change the controlled state and update the red LED.
- Continue printing the UID and the `GRANTED` or `DENIED` result on the Serial Monitor.

This extension combines Digital Output knowledge from previous labs with UID data from the RC522 Device.

The processing flow becomes:

RFID card → UID → Compare → Application state → LED + Serial Monitor

A complete access-control system can add an actuator or a mechanism for managing the UID list. Within the scope of Lab 10, the objective is only to use the UID as identification data to practice application-level decision structures [2].

7 References

- [1] NXP Semiconductors, *MFRC522 Standard performance MIFARE and NTAG frontend*, Rev. 3.9, 2016.
- [2] Anis Koubaa, *Lab: RFID Access Control System*, SE322: Internet of Things Applications, College of Engineering, Alfaisal University, 2025.
- [3] MaaZEDU, *MaaZEDU Development Board Guide*.

[4] EduFramework, *RC522 Device API*, EduFramework Source Code.